Université de Strasbourg

BDR THERMEA GROUP

University of Strasbourg

Project-based TU
Report

Multi-Physics Modeling

Students :
Iman Barkan
Job Congo

Supervisors :
Christophe Prud'Homme
Joubine Aghili
Didier Lemetayer
Paul Devigne
Hanae Mbarki

February 1, 2024

Contents

1 Introduction **2**
1.1 Context . . . 2
1.2 Objectives . . . 2
1.2.1 Modeling . . . 2
1.2.2 Data Analysis . . . 3

2 Operation of a Heat Pump **3**
2.1 Basic Principles and Mechanisms . . . 3
2.2 Thermodynamic Cycle of an HP . . . 3
2.3 Key Components of the HP . . . 4

3 Heat Pump Model **5**
3.1 FMU Parameters . . . 5
3.2 FMU Inputs . . . 6
3.3 FMU Outputs . . . 7
3.4 Simulation of the FMU . . . 8

4 Data presentation **9**

5 Calibration Algorithms **10**
5.1 Non-dominated Sorting Genetic Algorithm (NSGA-II) . . . 10
5.1.1 Overview . . . 10
5.1.2 NSGA-II algorithm stages . . . 10
5.2 Particle Swarm Optimization (PSO) . . . 11
5.2.1 Overview . . . 11
5.2.2 Algorithmic Principles . . . 11
5.2.3 Pyswarm PSO Implementation . . . 13
5.2.4 Calibration using PSO . . . 13

6 Implementation **14**
6.1 Line by Line Calibration using NSGA-II . . . 14
6.2 Line by Line Calibration using PSO . . . 15

7 Results **16**
7.1 Optimal Parameters . . . 16
7.2 Simulation vs. Expected Results . . . 17
7.3 Conclusive Assessment . . . 17

8 Conclusion **18**

1 Introduction

1.1 Context

In a world where energy efficiency and sustainable solutions are becoming increasingly important, our project, proposed in collaboration with BDRThermea, aims to explore new avenues in understanding and optimizing heat pumps. BDRThermea, a renowned player in the heating and cooling industry, is recognized for its deep commitment to sustainability and innovation. Founded on a mission of decarbonization, the company stands out for its proactive approach and ongoing investment in research and development. These systems, essential in heating and cooling applications, represent a fertile ground for technical and ecological innovation.

The concept of reverse engineering is at the heart of our approach. This technique, which involves deeply analyzing an existing system to deduce the underlying processes and parameters, is a powerful tool for improvement, innovation, and understanding. In our project, reverse engineering is applied to a heat pump compressor, a key component whose performance directly influences the overall efficiency of the system. The project benefits from close collaboration with the CAE & Modeling team of BDRThermea, an R&D division created in May 2022. This team is a group of specialists dedicated to helping develop innovative products and supporting all other hubs in four key areas: flow (fluid mechanics), multiphysics modeling, mechanics, and acoustics.

Our project focuses on a specific heat pump, whose steady-state performance has been previously tested and documented. However, although the pump itself has been modeled as a set of different components, the specific parameters of the compressor model remain unknown. Our main objective is therefore to fill this gap, by identifying and determining the missing parameters of the compressor model. This approach is essential not only for a complete understanding of the heat pump's operation but also for potential future improvements in its design and performance.

1.2 Objectives

The project aims to deepen understanding and optimization of heat pumps, focusing on the reverse engineering of key components, particularly the compressor. Our temporary roadmap revolves around several crucial steps, aiming to cover the entire modeling and analysis process.

1.2.1 Modeling

- Understand the operation of heat pumps through bibliographic research and training. This initial phase is essential to acquire a solid foundation on which to base our subsequent work.
- Examine the technical documentation of compressors, focusing on the types of models already existing and their relevance to our project. This involves a thorough identification of the parameters used by BDRThermea for rotary compressors.

1.2.2 Data Analysis

- Data Extraction and Preparation: Clean and prepare the data from the entire heat pump system, eliminating errors or inconsistencies, to ensure the reliability of future analyses.
- Parameter Calibration: Use test data to calibrate the parameters. This includes a sensitivity analysis to assess the impact of each input parameter on the performance of the heat pump.
- Cross-Validation: Compare the predictions of the calibrated model with a separate test dataset to verify the accuracy and reliability of the model.

2 Operation of a Heat Pump

2.1 Basic Principles and Mechanisms

The heat pump (HP) is a key element in the landscape of modern heating and cooling systems, characterized by its ability to transfer thermal energy from a low-temperature environment to a warmer medium. This technology defies the spontaneous transfer of heat, thus achieving a reversal of the natural flow of thermal energy. It stands out for its versatile role, serving as a heating system when it raises the temperature of a hot source, or acting as a refrigeration system by lowering the temperature of a cold source. The applications of the HP encompass various fields, from domestic air conditioners to industrial refrigerators.

The operation of an HP is similar to that of a refrigerator but with an inverted process for heating. Instead of extracting heat from a confined space to cool it, it captures heat from the external environment and transfers it inside a building. This process relies on the compression and expansion of a refrigerant gas, typically fluids like R407C, R410A, R134a, or R32, following precise thermodynamic cycles.

2.2 Thermodynamic Cycle of an HP

An HP operates according to a transformation cycle in four key stages, involving phase changes of the refrigerant:

Compression: In this phase, the refrigerant, initially in vapor form, is compressed, causing an increase in its pressure and temperature.

Condensation: The refrigerant, now at high temperature and pressure, passes through a condenser. Here, it releases its heat to the surrounding medium (e.g., the air in a room) and transforms into a liquid.

Expansion: After condensation, the high-pressure liquid passes through an expander. This step rapidly reduces the fluid's pressure, causing partial vaporization of it and resulting in a decrease in its temperature.

Evaporation: The refrigerant, now cold and partially vaporized, circulates through a heat exchanger (evaporator) located in the area to be cooled. By absorbing heat from this environment, the fluid completely evaporates, returning to the gaseous state.

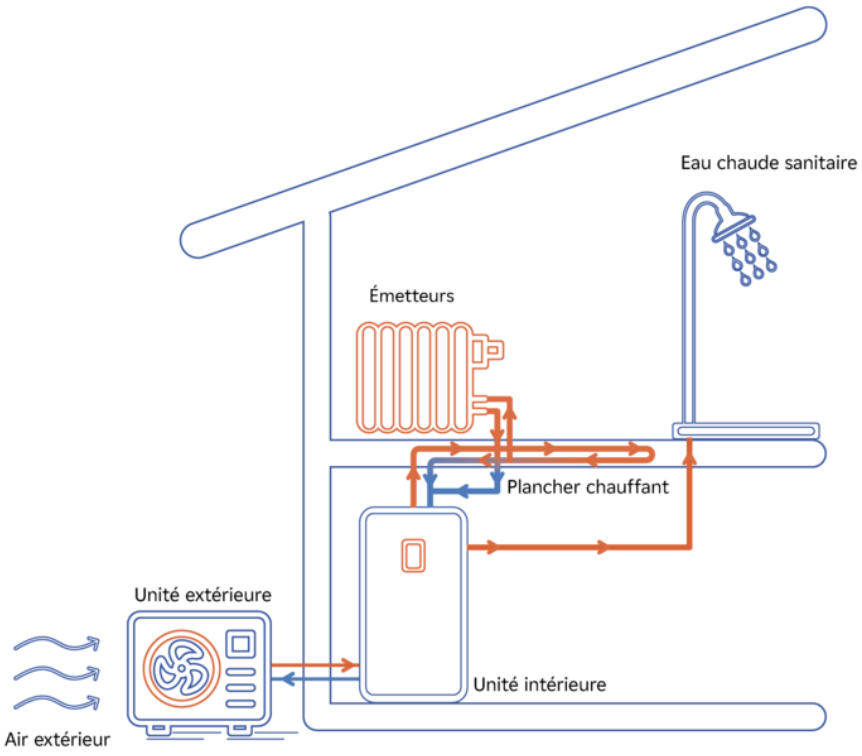

Figure 1: Operating diagram of HP [6]

2.3 Key Components of the HP

The circuit of the HP consists of four main components, each playing a vital role in the cycle:

The Condenser (Hot Source): Here, the refrigerant releases its heat to the secondary fluid (water, air, etc.), changing from gaseous to liquid state.

The Expander: This component reduces the pressure of the liquid refrigerant.

The Evaporator (Cold Source): Heat is absorbed from the secondary fluid, vaporizing the refrigerant.

The Compressor: Powered electrically or mechanically, it increases the pressure and temperature of the gaseous refrigerant.

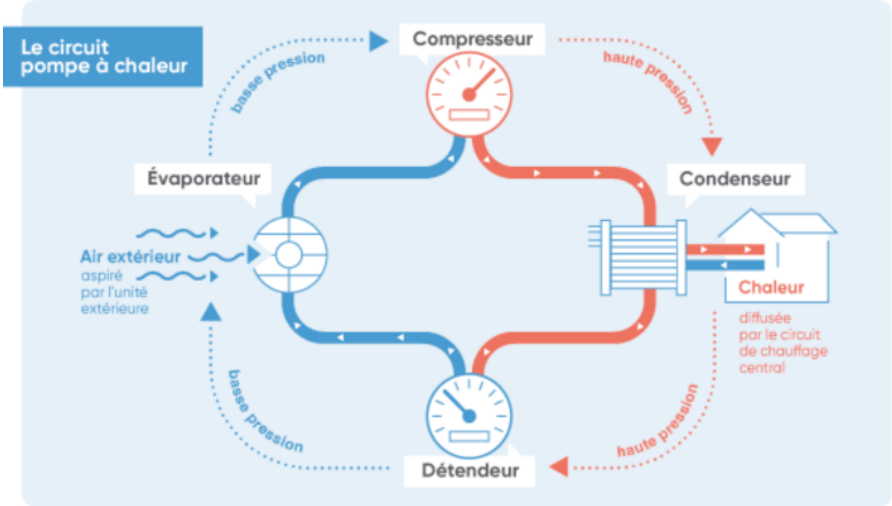

Figure 2: Component diagram of HP [5]

3 Heat Pump Model

3.1 FMU Parameters

In the modeling of the heat pump, various parameters are adjustable within their respective ranges. These parameters are crucial in simulating different operating conditions and analyzing the compressor's behavior under varied scenarios. The modifiable parameters include:

- **x_areaLeakage**: This parameter represents the total area through which leakage can occur within the compressor. The range from 1×10^{-8} to 1×10^{-6} square meters allows for the simulation of various leakage scenarios, which could be due to manufacturing imperfections, wear and tear, or design specifications. Leakage in a compressor can lead to a decrease in efficiency as it allows some of the refrigerant to escape the compression cycle, thus reducing the overall effectiveness of heat transfer.

- **x_areaSuctionValve**: The area of the suction valve, adjustable between 2×10^{-6} and 1×10^{-3} square meters, plays a critical role in controlling the flow of refrigerant into the compressor. A larger valve area would typically allow more refrigerant to enter the compressor, potentially increasing the cooling capacity but also possibly affecting the efficiency and stability of the cycle. Conversely, a smaller valve area restricts the refrigerant flow, which could lead to lower power consumption but reduced cooling capacity.

- **x_areaDischargeValve**: This parameter corresponds to the discharge valve area, which controls the flow of compressed refrigerant out of the compressor. The range from 1×10^{-7} to 1×10^{-4} square meters allows for the investigation of how the size of this valve affects the performance of the compressor. The discharge valve's area impacts the pressure within the compressor and influences the refrigerant's flow rate into the condenser, playing a crucial role in the efficiency and operational stability of the heat pump.

- **x_relativeDeadSpace**: This parameter, varying between 0.001 and 0.1, reflects the relative dead space in the compressor. Dead space is the volume within the compressor where the refrigerant does not contribute to the compression process. This non-active volume can affect the efficiency of the compressor, as a larger dead space usually leads to lower efficiency. It represents the volume where the refrigerant is trapped and recompressed in each cycle without contributing to the cooling or heating process.

- **x_driveEfficiency**: This parameter indicates the efficiency of the compressor's drive, spanning from 0.85 to 0.98. It represents how effectively the drive (usually an electric motor) converts electrical energy into mechanical energy to drive the compressor. Higher efficiency means that more of the electrical energy is converted into useful work for compressing the refrigerant, leading to a more efficient heat pump system overall. This parameter is especially critical in determining the overall energy consumption and performance of the heat pump.

The **x_driveEfficiency** parameter is particularly significant as it corresponds to the efficiency of the electronic component regulating the compressor's frequency. Its dependency on the compressor frequency is modeled through a polynomial expression of the fourth order:

$$x_{\text{driveEfficiency}} = a_5 \times \text{freq}^4 + a_4 \times \text{freq}^3 + a_3 \times \text{freq}^2 + a_2 \times \text{freq} + a_1 \tag{1}$$

where freq represents the compressor frequency, and a_1, a_2, a_3, a_4, a_5 are the coefficients of the polynomial.

3.2 FMU Inputs

The Functional Mock-up Unit (FMU) for the heat pump model accepts several critical inputs, each playing a key role in simulating the system's behavior:

- **u_compressorFrequency (Compressor Frequency)**: This input represents the operating frequency of the compressor, measured in Hertz (Hz). The compressor frequency is a pivotal factor influencing the rate at which the refrigerant is compressed and circulated within the system. Higher frequencies typically increase the rate of compression and circulation, which can affect the cooling or heating capacity and the overall efficiency of the heat pump.

- **u_VFlow (Volumetric Water Flow)**: This input specifies the volumetric flow rate of water through the heat pump, measured in cubic meters per second (m^3/s). The water flow rate is crucial in determining the heat transfer rate within the heat pump, as it affects how quickly heat is absorbed or released by the water circulating through the system.

- **u_T_water (Water Inlet Temperature)**: This input denotes the temperature of water entering the heat pump, measured in Kelvin (K). The inlet water temperature is an essential parameter in the heat pump's thermodynamic cycle, influencing the efficiency and the amount of heat absorbed or released during the process.

- **u_T_air (Air Inlet Temperature)**: This input measures the air temperature at the heat pump inlet, also in Kelvin (K). The air temperature is a critical environmental factor affecting the heat pump's performance, especially in air-source heat pumps, where the air serves as either the heat source or sink.

3.3 FMU Outputs

The outputs from the FMU provide insights into the performance and efficiency of the heat pump under simulated conditions:

- **y_T_out (Water Outlet Temperature)**: This output reflects the temperature of water leaving the heat pump, measured in Kelvin (K). It is indicative of the heat pump's ability to transfer heat and is crucial for assessing the system's heating or cooling performance.

- **y_heatPower (Thermal Power)**: This output, measured in Watts (W), represents the thermal power generated on the hot side of the heat pump. It indicates the rate at which the heat pump transfers heat, which is vital for evaluating its heating performance and efficiency.

- **y_elecPower (Electrical Power Consumption)**: This output measures the electrical power consumption of the heat pump in Watts (W). It is a key indicator of the system's energy efficiency, representing the electrical energy required to operate the heat pump.

- **y_COP (Coefficient of Performance)**:

  The Coefficient of Performance is a critical output from the Functional Mock-up Unit (FMU), offering a dimensionless measure of the heat pump's efficiency. It is computed based on the thermodynamic principles governing the heat pump's operation. The COP varies depending on whether the heat pump is in cooling or heating mode, as the interest shifts between the cold and hot reservoirs. The equation for COP is expressed as [2]:

  $$COP = \frac{|Q|}{W} \tag{2}$$

  where:

  -Q represents the useful heat supplied (in heating mode) or removed (in cooling mode) by the system.

  -W is the net work input into the system for one cycle, which is always positive.

  The COP values for cooling and heating differ due to the distinct focus on either the cold or hot reservoir:

  -**Cooling COP**: In cooling mode, the COP calculates the efficiency in terms of the heat removed from the cold reservoir relative to the input work. It is defined as:

  $$COP_{cooling} = \frac{|Q_C|}{W} = \frac{Q_C}{W} \tag{3}$$

  Here, Q_C is the heat absorbed from the cold reservoir.

-**Heating COP**: Conversely, in heating mode, the COP is the ratio of the magnitude of heat delivered to the hot reservoir to the input work. Since the hot reservoir receives the heat extracted from the cold reservoir plus the input work, the heating COP is formulated as:

$$COP_{heating} = \frac{|Q_H|}{W} = \frac{Q_C + W}{W} = COP_{cooling} + 1 \tag{4}$$

Here, Q_H is the heat released to the hot reservoir, which is negative as per the convention in thermodynamics.

3.4 Simulation of the FMU

The `HeatPumpSimulator` class in Python offers a framework for simulating the heat pump's performance using the FMU2Slave interface from the FMPy library. The class is designed to streamline the simulation process, allowing for a detailed analysis of the heat pump under various conditions. Its functionalities include:

- **Initialization**: The constructor initializes the class with the FMU file name, simulation start and stop times, and the step size. It reads the model description from the FMU file using FMPy's `read_model_description` function. This step is crucial as it sets the stage for the entire simulation by loading the necessary model information and defining the simulation timeline.
- **FMU Preparation**: The `prepare_fmu` method handles the extraction and preparation of the FMU file for simulation. It locates the FMU file within the notebook directory, creates an `unzip` directory for the extracted contents, and uses the `extract` function from FMPy to unpack the FMU. This method ensures that all necessary files and components of the FMU are accessible for the simulation.
- **Parameter Range Verification**: The `verify_parameter_ranges` method checks if the initial values of the simulation parameters are within their defined permissible ranges. This validation step is crucial for maintaining the integrity and accuracy of the simulation, ensuring that the parameters used are realistic and within expected bounds.
- **Timeout-Enabled Simulation**: The `simulate_with_timeout` method runs the simulation within a separate thread, allowing for a timeout mechanism. This approach prevents the simulation from running indefinitely in case of unresponsive scenarios or computational issues, thereby enhancing the robustness of the simulation framework.
- **Simulation Execution**:

  -The `simulate` method is the core of the simulation process. It instantiates an FMU slave using the model's GUID and model identifier, then sets up the simulation with the given start values.

  -The setup involves initializing the FMU for the experiment, setting the real values of the parameters, and then entering and exiting the initialization mode.

-The `run_simulation` sub-method then executes the simulation over the defined timeframe. It iterates through the simulation time, performing steps at each interval and collecting output values for each of the specified variables.

-Exception handling is incorporated to ensure any errors during simulation are caught and handled appropriately, preventing crashes and providing meaningful error messages.

- **Data Collection and Analysis**:

  -As the simulation runs, data for each output variable (like `y_T_out`, `y_heatPower`, `y_elecPower`, `y_COP`) is collected and stored in a dictionary format.

  -This data can be used for further analysis, such as plotting the results to visualize the performance of the heat pump over time. While the `plot_results` method is a placeholder in the provided code, it can be implemented to generate graphs and charts for a more intuitive understanding of the simulation results.

4 Data presentation

The data represents the results of 85 tests carried out on a heat pump. The information is organised in columns with various characteristics recorded during the tests. After deleting incomplete results, there are 79 usable data items. Here is a general description of the data :

General information:

- Date: Test date
- Test location : Mertzwiller or Techneco
- ODU : Type of system (GP561)
- P.H.E.: Model (B26Hx26)
- Réfrigérant : R32
- Charge : 980 g

Main features:

- Setpoint : run times in seconds.
- Frequency : Compressor frequency in Hz
- Water in : Water temperature at heat pump inlet °C
- Water out : Water temperature at heat pump outlet °C
- Flow : Water flow rate in m^3/h
- Defrosts : Présence ou absence de dégivrage (yes/no)
- Current (max) : Maximum electrical current

- Power out : Heat pump thermal power in W
- Power in : Electrical power of the heat pump in W
- COP : coefficient of performance of the heat pump

Additional features:

- Toshiba PCBs : Presence of Toshiba PCBs (yes/no)
- Toshiba PCBs : Presence of main board (yes/no)
- Pump : Presence of the pump (yes/no)
- Corrected COP : COP corrected according to certain conditions
- Remarks : Observations or specific notes
- rotation : rotation of heat pump in hz

5 Calibration Algorithms

5.1 Non-dominated Sorting Genetic Algorithm (NSGA-II)

5.1.1 Overview

The NSGA-II (Non-Dominated Sorting Genetic Algorithm II) algorithm is an evolutionary technique developed for solving multi-objective optimisation problems. It was introduced by Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and T. Meyarivan in 2002, as an improvement on the original NSGA.

The main objective of the NSGA-II algorithm is to find solutions that are not dominated by other solutions, by providing a set of optimal and balanced solutions in the objective space.

5.1.2 NSGA-II algorithm stages

The various stages in the general operation of the NSGA-II algorithm are as follows.

1. Random creation of the first generation P_0 of size N, from the imposed search spaces.
2. Creation of a set of children Q_t of size N by genetic operators such as selection, crossover, and mutation.
3. Mixture of P_t and Q_t: $R_t = Q_t \cup P_t$.
4. Calculation of all the boundaries F_i of R_t and addition into P_{t+1} until the size of P_{t+1} is equal to N.
5. Returns to 2.

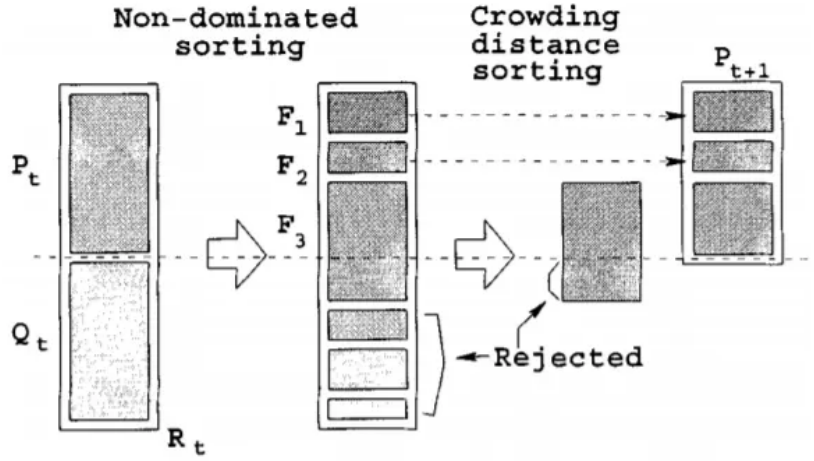

Figure 3: NSGA-II algorithm stages [7]

Non-dominated sorting: NSGA-II uses a non-dominated sorting technique to classify solutions into multiple edges, where solutions on the first edge are not dominated by any other solution, solutions on the second edge are dominated only by those on the first edge, and so on.

Crowding distance sorting : For each solution in the F_3 front, the crowding distance is calculated by considering the values of the neighbouring criteria. More precisely, the crowding distance of a solution is the sum of the distances between neighbouring solutions on each of the criteria.

5.2 Particle Swarm Optimization (PSO)

5.2.1 Overview

Particle Swarm Optimization (PSO) is a computational method that optimizes a problem by iteratively improving a candidate solution with regard to a given measure of quality. PSO utilizes a swarm of particles, which are potential solutions, to probe the search space. Particles adjust their positions in the search space by following two essential components: the cognitive and social components. Introduced by James Kennedy and Russel Eberhart in 1995 [1], PSO's inspiration stems from the social behavior patterns of animals such as bird flocking and fish schooling. These natural processes led to PSO being categorized under Swarm Intelligence and Artificial Life domains. The algorithm's strength lies in its simplicity and minimal need for parameterization.

5.2.2 Algorithmic Principles

The core of PSO lies in simulating the movements of a swarm of particles through the problem space. Each particle represents a potential solution to the optimization problem. The particles 'fly' through the problem space and adjust their positions based on their own experience and the experience of neighboring particles.

Particle Dynamics Each particle in the swarm has two main attributes:
- **Position** (x_i): Represents the current solution of the particle.

- **Velocity** (v_i): Dictates the direction and magnitude of the particle's movement in the search space.

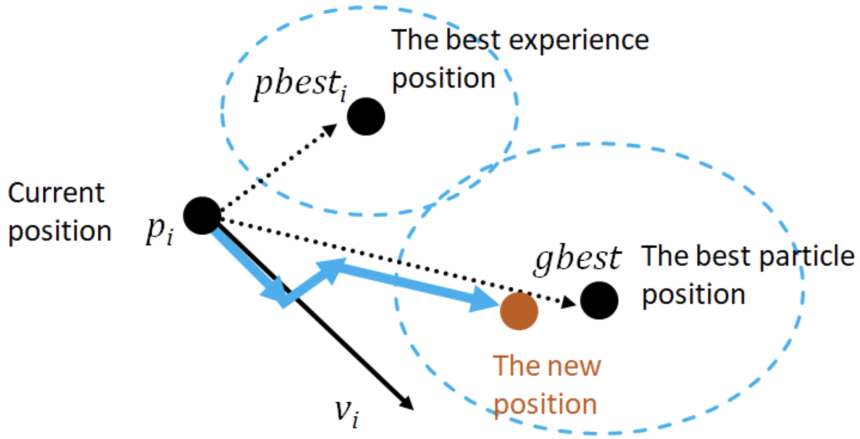

Figure 4: Position and Velocity Vectors [3]

Position and Velocity Update The position and velocity of each particle are updated according to the following equations:

$$x_{i,d}(t+1) = x_{i,d}(t) + v_{i,d}(t+1) \tag{5}$$

$$v_{i,d}(t+1) = v_{i,d}(t) + C_1 \cdot \text{Rnd}(0,1) \cdot [p_{bi,d}(t) - x_{i,d}(t)] + C_2 \cdot \text{Rnd}(0,1) \cdot [g_{b,d}(t) - x_{i,d}(t)] \tag{6}$$

Where:

- $x_{i,d}(t)$ and $v_{i,d}(t)$ are the position and velocity of particle i in dimension d at iteration t.
- C_1 and C_2 are cognitive and social acceleration constants.
- Rnd(0, 1) is a random number between 0 and 1.
- $p_{bi,d}(t)$ is the best-known position of particle i in dimension d.
- $g_{b,d}(t)$ is the best-known position among all particles in the swarm.

Cognitive and Social Components - The cognitive component (C_1) represents the particle's own experience. It is the knowledge of the particle's best position found so far.

- The social component (C_2) represents learning from the swarm's experience, reflecting the best position found by any particle in the swarm.

Initialization and Control Parameters The initialization of the swarm is critical to the algorithm's performance. Particles are typically placed randomly or uniformly within the problem space, with velocities often initialized to small random values. Key parameters in PSO include:

- Swarm size.
- Values of C_1 and C_2, which balance exploration and exploitation.
- Maximum iterations or convergence criteria.

5.2.3 Pyswarm PSO Implementation

The Particle Swarm Optimization algorithm, as implemented in the Pyswarm Python library [4], is a method for finding optimal solutions to complex problems. The code is structured to utilize the PSO algorithm for calibration purposes, optimizing parameters based on a given objective function.

Initialization and Parameter Setting The function `pso` accepts the objective function (`func`), lower (`lb`) and upper (`ub`) bounds of the search space, and various PSO-specific parameters. The lower and upper bounds define the feasible region within which the particles can search for solutions.

Key parameters for the PSO include `swarmsize` (number of particles in the swarm), `omega` (inertia weight), `phip` (personal learning coefficient), and `phig` (global learning coefficient). These parameters govern the behavior and efficiency of the swarm.

Particle Dynamics Each particle in the swarm is represented by its position and velocity. The position (x_i) indicates the current solution of the particle, and the velocity (v_i) determines the particle's movement through the search space.

The position and velocity of each particle are updated iteratively based on their personal best position, the swarm's global best position, and random factors that encourage exploration. The update equations are as follows:

$$v_{i,d} = \omega \cdot v_{i,d} + \text{phip} \cdot \text{Rnd}(0,1) \cdot (p_{bi,d} - x_{i,d}) + \text{phig} \cdot \text{Rnd}(0,1) \cdot (g_{b,d} - x_{i,d}) \quad (7)$$

$$x_{i,d} = x_{i,d} + v_{i,d} \quad (8)$$

Objective Function Evaluation The core of the optimization process involves evaluating the fitness of each particle using the objective function `func`. This function assesses the quality of each solution, guiding the swarm towards the most promising regions of the search space.

5.2.4 Calibration using PSO

The `line_calibration` function implemented in our simulation notebook demonstrates the application of PSO for calibration. It optimizes parameters for each line in the input DataFrame `df_input_no_defrost_cal`, using the PSO algorithm to minimize the objective function.

- The function iterates over each line of the DataFrame.
- For each line, PSO is executed by calling the `pso` function with the objective function and the bounds (`lb` and `ub`).
- The best solution (`xopt`) and the corresponding error (`fopt`) for each line are stored and displayed.
- The results for all lines are saved using the `save_results` function.

6 Implementation

To preface the detailed implementation of the calibration process, it's important to note that the entire codebase is structured into two separate notebooks, each dedicated to a specific calibration method. This division ensures a focused approach to applying and understanding each calibration technique independently. One notebook addresses the calibration using the NSGA-II algorithm, while the other deals with the Particle Swarm Optimization (PSO) method. This setup allows for a clear and organized way to execute, analyze, and compare the effectiveness of each method in the calibration process.

6.1 Line by Line Calibration using NSGA-II

The 4 objective functions `objective_T_out`,`objective_heatPower`,`objective_elecPower`, `objective_COP` calculate an error by taking the absolute value of the difference between the simulated results and the real data. These functions take a set of parameters and a row index, representing a specific line of data.

Then we call `apply_NSGA2`. This function takes as input a parameter set for calibration. It uses NSGA-II to optimise a problem defined by objective functions (`objective_T_out`,`objectiv` `objective_COP`). The results of the optimisation, i.e. the optimal solutions and the values of the corresponding objective functions, are stored in a file.

```
def apply_NSGA2(df_input_no_defrost_cal, pop_size=20, n_gen = 15):

    ...
    # Define the problem
    my_problem= FunctionalProblem(n_var=5,objs = objs,
                          xl=np.array([1e-8, 2e-6,1e-7 , 0.001, 0.85]),
                          xu=np.array([1e-6,1e-3,1e-4 ,0.1,0.98]))

    # Optimize the problem using NSGA-II
    algorithm = NSGA2(pop_size=pop_size)
    result = minimize(my_problem,
                  algorithm,
                  ('n_gen', n_gen),
                  seed=1,
                  verbose=True)

    ...
```

6.2 Line by Line Calibration using PSO

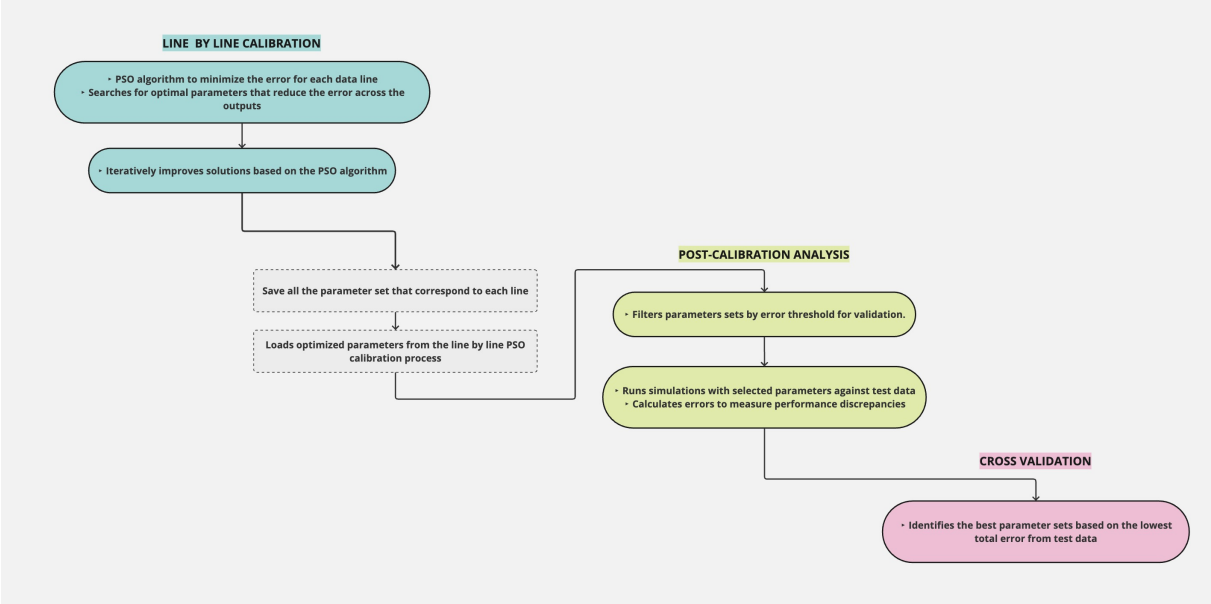

Figure 5: Overview of the calibration process

The heart of our calibration process is the `objective_function`, which evaluates the fitness of a parameter set. This function takes a set of parameters and a line index, representing a specific data row, and computes an error based on the difference between simulated results and actual data.

```
def objective_function(params, line_index=0):
    ...
    simulation_result = simulator.simulate_with_timeout(start_values, timeout
        =10)
    ...
    error = np.sum((last_row[['y_T_out', 'y_heatPower', 'y_elecPower', 'y_COP']]
         -
                    [output_row['...']])**2)
    ...
```

Here, the simulation result for a given row from the dataset is compared against the actual output, and a sum of squared differences across multiple outputs (`y_T_out`, `y_heatPower`, `y_elecPower`, `y_COP`) is computed as the error.

The calibration procedure involves running the PSO algorithm on each line of the input DataFrame. For each line, the PSO algorithm searches for the set of parameters that minimizes the error computed by the `objective_function`.

```
def line_calibration(df_input_no_defrost_cal, swarm_size, max_iter):
    ...
    for line_index in range(len(df_input_no_defrost_cal)):
        xopt, fopt = pso(lambda params: objective_function(params, line_index),
                         lb, ub, swarmsize=swarm_size, maxiter=max_iter)
        ...
```

The `pso` function is called with the objective function, bounds (lower `lb` and upper `ub`), swarm size, and maximum iterations. The best parameters (`xopt`) and the corresponding minimal error (`fopt`) are captured for each line.

After running the PSO calibration, a JSON file containing the optimized parameters is loaded. Parameters resulting in an error below a certain threshold (e.g., 500) are filtered for further validation.

```
filtered_params = {k: v for k, v in optimized_params.items() if v['fopt'] < 500}
```

The selected parameters are then tested against a separate dataset to evaluate their performance.

For each set of filtered parameters, the simulation is run against test data, and errors are calculated using the `calculate_error` function, which assesses the discrepancy between simulated and expected results.

```
for key, params in filtered_params.items():
    ...
    for index, input_row in df_input_no_defrost_test.iterrows():
        ...
        simulation_results = simulator.simulate_with_timeout(start_values)
        total_error, errors = calculate_error(simulation_results,
            df_output_no_defrost_test.iloc[index])
        ...
```

The process involves iterating over the test dataset, running the simulation with the selected parameters, and calculating the total error. The parameters that yield the lowest error across the test dataset are deemed the best.

7 Results

7.1 Optimal Parameters

Through the application of Particle Swarm Optimization (PSO), we have systematically calibrated our heat pump model to closely mirror the actual operational data. The optimal parameters derived from our extensive simulations are as follows:

- Leakage Area: 1×10^{-8} $\mathbf{m^2}$
- Suction Valve Area: $5.724439745087251 \times 10^{-5}$ $\mathbf{m^2}$
- Discharge Valve Area: $4.353043641598575 \times 10^{-5}$ $\mathbf{m^2}$
- Relative Dead Space: 0.09933469072929339 (dimensionless)
- Drive Efficiency: 0.8963402568638703 (dimensionless)

These parameters were pinpointed to be the most effective in minimizing the error between simulated and real-world measured values.

7.2 Simulation vs. Expected Results

An line-by-line evaluation was conducted using the identified optimal parameters, comparing the model's simulated output with actual operational data. The outcomes are encapsulated in the following figures, which illustrate the percentage error between the expected and simulated results for key performance metrics:

- **Water Outlet Temperature (y_T_out)**: Errors ranged narrowly, indicating high accuracy in temperature-related predictions.
- **Heat Power (y_heatPower)**: Showed a moderate variance, underscoring the model's reliability in estimating thermal power output.
- **Electrical Power Consumption (y_elecPower)**: Demonstrated a close alignment with the actual power usage of the system.
- **Coefficient of Performance (y_COP)**: Maintained a consistent accuracy, reflecting the efficiency of the heat pump effectively.

The graphical analysis, plotted with error thresholds of $\pm 5\%$ and $\pm 10\%$, revealed that the majority of the simulated results fell within acceptable error margins, asserting the model's validity (See Figure 6).

7.3 Conclusive Assessment

The presented results validate the effectiveness of the PSO-based calibration method, as it has enhanced the precision of our heat pump model. The calibration approach not only optimized the parameters but also provided a clear insight into the model's predictive power and its limitations. Future work may include refining the model further and expanding its applicability across different heat pump configurations and operational conditions.

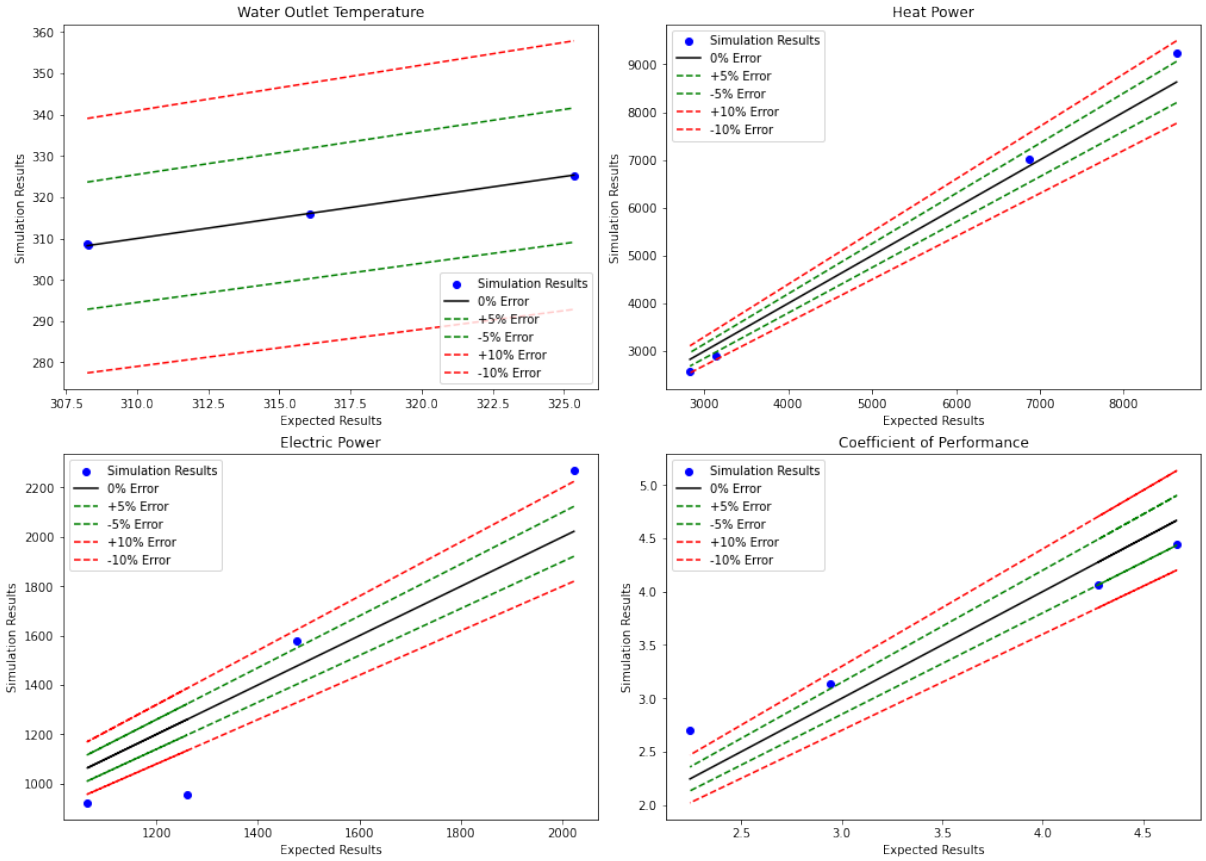

Figure 6: Percentage errors in Water Outlet Temperature, Heat Power, Electrical Power, and Coefficient of Performance between simulation and expected results.

8 Conclusion

In conclusion, while our calibration methods have shown promise, especially with the PSO algorithm yielding tangible results, our journey encountered significant challenges. Notably, we aimed to incorporate an uncertainty analysis using OPENTURNS to enhance our calibration process further. Such an analysis would have been instrumental in identifying which performance indices were most sensitive to parameter variations, thus allowing us to focus our efforts more efficiently. Unfortunately, due to unforeseen computer bugs and the constraints of time, we were unable to implement this aspect of our study. This limitation prevented us from potentially achieving a more targeted and effective calibration process.

Despite this setback, it's important to highlight the achievements made. The PSO algorithm, in particular, has proven to be effective, albeit with room for further refinement. The calibration process, which processed all lines of heat pump data, was an exhaustive task. It took approximately 10 hours, with a swarm size of 8 and a maximum of 18 iterations, to complete. This underscores the intensive computational effort required but also points to the potential for optimizing these processes further.

Although the NSGA-II algorithm did not meet our expectations in terms of effectiveness, the experience gained from employing these diverse calibration methods has been invaluable. Moving forward, it would be beneficial to address the technical issues encountered with OPENTURNS and to continue refining our calibration approach. By doing so,

we can enhance the accuracy and reliability of our simulations, ultimately contributing to more efficient and effective heat pump system designs.

References

[1] G. Pereira, *Particle swarm optimization*, https://www.researchgate.net/publication/228518470_Particle_Swarm_Optimization, Accessed: 2024-01, Apr. 2011.

[2] M. Chesser, P. Lyons, P. O'Reilly, and P. Carroll, "Air source heat pump in-situ performance," *Energy and Buildings*, vol. 251, p. 111 365, 2021. DOI: https://doi.org/10.1016/j.enbuild.2021.111365. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0378778821006496.

[3] Baeldung, *Particle Swarm Optimization Vectors*, https://www.baeldung.com/wp-content/uploads/sites/4/2022/10/pso-vectors.png, Accessed: 2024-01, 2022.

[4] Timothy Reluga, *pso.py - Particle Swarm Optimization Implementation in PySwarm*, https://github.com/tisimst/pyswarm/blob/master/pyswarm/pso.py, Accessed: 2024-01, 2022.

[5] Atlantic, *Quels sont les composants d'une pompe à chaleur?* https://www.atlantic.fr/aide/pompe-a-chaleur/fonctionnement/quels-sont-les-composants-d-une-pompe-a-chaleur/, Accessed: 2024-01.

[6] IZI, *Comment fonctionne une pompe à chaleur ?* https://www.izi-by-edf-renov.fr/blog/fonctionnement-pompe-a-chaleur, Accessed: 2024-01.

[7] medium, https://medium.com/@rossleecooloh/optimization-algorithm-nsga-ii-and-python-package-deap-fca0be6b2ffc, Accessed: 2024-01.